



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۲۵ آذر

فصل پنجم: لایه *Network* بخش *Control – Plane* اسلاید ۹۰ تا ۱۱۷ [۱]

مدیریت شبکه

مدیریت شبکه از ۴ بخش تشکیل شده است:

۱. سرور مدیریت که تحت نظر مدیر سرور است.
۲. دستگاه‌های نرم افزاری و سخت افزاری قابل برنامه ریزی و مدیریت پذیر
۳. دیتا‌های دستگاه‌ها مثل وضعیت آنها و دیتا‌های پیکربندی و دیتا‌های عملکردی و آمار دستگاه‌های مختلف
۴. پروتکل مدیریت شبکه که قالب پیام‌های ارتباطی برای مواردی مثل پیکربندی و ارسال تنظیمات و مدیریت دستگاه‌ها توسط مدیر سرور است. یا برای مواردی که دستگاه‌ها پیشامد یا داده‌ای را می‌خواهند به سرور مدیر بدهند.

نحوه مدیریت اپراتور روی شبکه

۱. استفاده از *CLI = command line interface* یعنی مدیر سرور از طریق مثلا *ssh* به دستگاه‌ها دسترسی و تنظیمات را روی تک‌تک آنها انجام دهد.
 ۲. پروتکل *SNMP* با کمک دیتابیس مدیریتی *MIB* داده‌هایی را پرس و جو میکند و یا داده‌هایی را تنظیم میکند
 ۳. *NETCONF* در سطح کل شبکه برای مدیریت چندین دستگاه. در سطح بالاتر و *abstract* تر و جامع‌تری استفاده میشود.
- از زبان مدل سازی داده‌ای *YANG* برای داده‌های رد و بدل شده استفاده میکنند.

پروتکل SNMP

نحوه انتقال اطلاعات با MIB در SNMP

سرور مدیر درخواست *request* خود از دستگاه مدیریت شونده را به پورت ۱۶۱ *UDP* روی سرور مدیریت شونده می‌فرستد. او هم پاسخ می‌دهد. اگر دستگاه مدیریت شونده دچار مشکل خاصی شود و به *exception* بخورد، روی مود *trap* قرار می‌گیرد و پیام *trap message* روی پورت ۱۶۲ *UDP* مدیر سرور ارسال می‌شود.

بخش‌های مختلف پروتکل SNMP

۱. *SMI* که شامل اسم آبجکت‌ها $OID = object\ identifier$ و تایپ آبجکت‌ها و همینطور شامل نحوه انکود کردن آنهاست. شبیه سینکس کد می‌شه
۲. *MIB* که یک مجموعه‌ای (مثل یک جدولی) از اسم آبجکت‌ها و نوعشون و نحوه ارتباط آنها با یکدیگر در یک *entity* می‌تواند باشد. شبیه *declaration*‌ها و تعریف‌ها و اینچیزا
۳. پروتکل *SNMP* که فرمت پیام‌های رد و بدل شده را بین مدیر و *Agent*‌ها مشخص می‌کند. مثلاً نحوه عملکرد توابع مختلف. شبیه خود *coding*.

SMI

در *SMI* هر کدام از آبجکت‌ها یک *OID* دارند که به نوعی ساختاری درختی دارد. در اسلاید ۹۶ ببین. علاوه بر آن، در اینجا نوع اشیاء و نحوه کد کردن اشیاء مشخص شده است. کد کردن آن از روی *tag, length, value* انجام می‌شود. توی اسلاید *tag* رو مثلاً *IPAddress* گرفته بود.

MIB

همانطور که گفتیم اطلاعات عملکردی و پیکربندی را در خود دارد. که در *MIB module* در هر دستگاه جمع‌آوری شده است.

مثلاً در جدول اسلاید ۹۷، می‌بینید که برای *udpTable* تایپش *SEQUENCE* است و به ازای هر پورت مورد استفاده در حال حاضر، یک مدخل وجود دارد.

بقیه معمولاً یک شمانده ۳۲ بیتی هستند. مثل متغیر *UDPInDatagrams* که تعداد کل دیتاگرام‌های تحویل شده را نشان می‌دهد یا *UDPNoPort* تعداد کل دیتاگرام‌هایی که تحویل مقصد نشده‌اند چون اپلیکیشنی در

پورت آن نبوده. یا مثلاً *UDInErrors* برای دیتاگرام‌هایی که با دلایل غیر از این، به مقصد نرسیده اند. یا متغیر *UDPOutDatagram* تعداد کل دیتاگرام‌هاییست که ارسال شده اند. همه این ۴ تا، شمارنده‌های ۳۲ بیتی هستند.

تایپ پیام‌های SNMP

مدیر به دستگاه با *get* برای دریافت اطلاعات و یا *set* برای تنظیم اطلاعات دستگاه‌ها. پیام دستگاه به مدیر یا پاسخ به درخواست مدیر و یا *trap* (برای آگاه‌سازی مدیر از رخ دادن *exception*) است.

قالب پیام‌های SNMP

برای پیام‌های نوع ۰ تا ۳ ساختار مشابه داریم مثلاً بخش اول هدر *get* و *set* قرار میگیرند. بقیه فیلدها هم با اعداد و نام‌های دیگر پر میشود. که همان *varibale*‌های توابع *get* و *set* است. در فیلد اول، تایپ *PDU type* قرار دارد که یکی از همان مقادیر ۰ تا ۳ در آن قرار میگیرد. در پیام‌های نوع ۴ که مربوط به *trap* است، هدر کمی متفاوت است. مثلاً فیلدی اینجا هست که نوع تایپ اون *trap* را مشخص میکند.

پیام‌ها به مدیر دیگر در SNMP

امکان این وجود دارد که یک مدیر با پیام‌های *Report* و *InformRequest* اطلاعاتی را با مدیرهای دیگر رد و بدل کند.

مراحل انجام پروتکل SNMP

در اسلاید ۱۰۲ آمده است و حتماً چک کن. ۶ مرحله دارد.

پروتکل NETCONF

این پروتکل برای مدیریت و پیکربندی شبکه در سطح جامع‌تری کاربرد دارد. و یک سری *action*‌هایش مثل بازبانی و اصلاح و تنظیم و فعال‌سازی پیکربندی‌هایش است. برخی از *action*‌هایش به صورت *atomic commit* است که روی چند دستگاه عمل میکند و این عمل‌ها غیر قابل تفکیک و وقفه هستند و *interrupt* در آنها رخ نمیدهد. امکان *subscribe* شدن برای مطلع شدن از اعلان‌های دستگاه‌ها وجود دارد. میتونیم پرسوجو بزنی و از داده‌های عملکردی و آماری با خبر شویم. در اینجا از پارادایم *RPC = remote procedure call* استفاده میکند به جای *P2P* یا *RTC* یا *CS* یا *publish - subscribe*

روی پروتکل انتقال مطمئن و امن مثل *TLS* منتقل میشود
پیام‌های آن به صورت *XML* کد میشوند.

مدل لایه‌ای *NETCONF*

۴ لایه داریم. لایه اول *content* لایه دوم برای *operation* لایه سوم *message* و لایه آخر برای *secure transport* است.

برای لایه *content* داده‌های پیکرد بندی و داده‌های *notification* هستند.
برای لایه دوم که *operation* است، توابعی مثل *get* یا *get – config* یا *edit – config* و غیره هستند.
در لایه سوم، پارادایم‌های مختلف مثل *rpc* و *notification* هستند که فرمت خاص انتقال پیام خود را دارند مثلاً *rpc* به صورت *xml*
و در لایه آخر یعنی چهارم هم *TLS* یا مثلاً کد *ssh* قرار میگیرد.

انواع *datastore* ها *NETCONF*

۱. *candidate*: میتوان آن را دستکاری کرد و روی پیکر بندی حال حاضر اثر ندارد.
 ۲. *running*: که همین انباره‌ای است که پیکربندی کامل و فعال دستگاه است و دستگاه آن را اجرا میکند.
 ۳. *startup*: پیکربندی است که در شروع دستگاه آن را لود میکند.
- با عمل کپی میتوان از پیکربندی *running* روی دو انباره دیگر کپی کرد یا میتوان پیکربندی *candidate* را روی *running* کامیت کرد.
مثلاً پیام‌های *rpc* و *reply – rpc* هر از چندگاهی بین مدیر سرور و *agent* مبادله میشود.
پیام *hello* و یا *close* دو طرفه انجام میشود.

بعضی از *Operation* های *NETCONF*

- *get – config*: برای گرفتن همه یا بخشی از *configuration* یک دستگاه. یک دستگاه میتواند چند تا پیکربندی داشته باشد.
- *get*: گرفتن و بازیابی اطلاعات مربوطه به پیکربندی و اطلاعات آماری و عملکردی
- *edit – config*: عوض کردن پیکربندی‌ها حتی پیکربندی‌های *running* از طریق *reply – rpc* یا *ok* یا *rpcerror* میگیرد و *rollback* میکند و عدم اجرا دستورات جدید از آخر به اول اجرا میکند؟
- *lock* و *unlock*: قفل کردن انباره تا نشود از طریق *CLI* یا *SNMP* یا *NETCONF* آن را تغییر داد. بعد آن را از قفل در میاورد

• *create – subscription* و *notification*: سیستم‌های مدیریت شونده برای فعال کردن اعلان *subscription* از آن استفاده میکنند.

مثلاً مشتری با *ssh* به دستگاه مورد نظر منتقل شده و با قابلیت‌های آن را پرس و جو میکند و دستگاه هم آن را اعلام میکند.

برای ادیت کردن انباره داده *candidate*

ابتدا آن را *clear* میکنیم و بعد آن را *edit* میکنیم و بعد آن را کامیت میکنیم. پس شد ۳ مرحله برای *edit – config* اگر بخواهیم بدون وقفه این کار انجام شود، اول یک *lock* میگیریم و بعد در آخر *unlock* میکنیم.

YANG

زبان مدل سازی داده است که برای مشخص کردن سیتکس و سمتیک و استراکچر داده‌های مدیریتی *NETCONF* به کار میرود. دارای نوع‌های داده پیش ساخته. *xml* را میتوان از روی توصیفات *YANG* تولید کرد. قیود مورد نظر میان داده‌ها را میتوان به کمک پیکر بندی آن برآورده کرد و میتوان مطمئن شد که قیود مربوط به پیکربندی و سازگاری و صحت آن‌ها را به درستی رعایت میکند.

مراجع

[1] “ch5-slide-90-to-117,” <https://drive.google.com/drive/folders/>, 2022.